

bigd_01_setup 퀴즈

빅데이터개론 — 관심종목 주가 알림 봇 | 학생용

I. 참거짓 (14 문항)

1. 매장에서 받는 압축파일에는 그 때까지의 완성된 코드가 들어 있다.
2. GitHub 웹에서 파일을 고치면 내 컴퓨터의 같은 파일도 함께 바뀐다.
3. 수집 → 저장 → 가공 → 시각화 → 확인 다섯 칸은 이 수업에서만 쓰는 학습용 구분이고, 회사에서 하는 데이터 일은 이와 다른 순서로 진행된다.
4. 저장소를 public 으로 만들면 나중에 private 으로 바꿀 수 없다.
5. .gitignore 파일 안에 적힌 파일은 GitHub 에 올라가지 않는다.
6. 윈도우에서 Python 을 설치할 때 Add python.exe to PATH 체크는 켜 두면 편한 선택 사항일 뿐이다.
7. VS Code 에서는 Python · Pylance · Python Debugger 확장을 각각 찾아 설치해야 한다.
8. 받은 압축파일은 학기말까지 계속 사용할 폴더이므로 위치를 정해두고 풀되, 폴더 경로에 한글이 섞이지 않게 한다.
9. 2 장에서 .env 를 만들었을 때 소스 제어의 커밋 대상 목록(Changes)에 .env 가 뜨지 않는 것이 정상이다.
10. git config 로 하는 이름·이메일 설정은 새 프로젝트를 만들 때마다 다시 해야 한다.
11. 소스 제어 패널 위쪽의 Initialize Repository 를 누르면 GitHub 에도 저장소가 함께 만들어진다.

12. 저장소 이름 입력칸에는 한글 폴더 이름이 그대로 보이지만, 실제로 만들어지는 이름은 한글이 전부 하이픈(-)으로 바뀐 것이다.

13. 저장소 이름을 친 뒤 그냥 엔터를 치면 public 저장소가 만들어진다.

14. Commit 버튼을 누르면 그 순간 GitHub 에도 파일이 올라간다.

II. 객관식 (28 문항)

15. 13 장 뒤 최종 결과물이 아닌 것은?

- ① 텔레그램으로 오는 현재가 알림
- ② 주식 자동 매매
- ③ 웹 대시보드

16. 완성된 결과를 받아 보는 두 경로를 바르게 짝지은 것은?

- ① push — 텔레그램 / pull — 웹 대시보드
- ② 둘 다 텔레그램으로만 온다
- ③ push — 웹 대시보드 / pull — 텔레그램

17. "원격과 로컬은 별개의 사본"이 뜻하는 것은?

- ① 한쪽을 바꿔도 다른 쪽은 그대로다
- ② 로컬 파일은 GitHub 에 자동 저장된다
- ③ 두 곳의 파일이 항상 똑같이 유지된다

18. 커밋(commit)에 가장 가까운 설명은?

- ① 파일을 삭제하는 것
- ② 프로그램을 실행하는 것
- ③ "여기까지 됐다"고 기록을 남기는 것

19. 막혔을 때 가장 먼저 할 일은?

- ① 코드를 처음부터 다시 받는다

- ② FAQ.md 2 절에서 오류 메시지를 검색한다
- ③ 다음 주까지 기다린다

20. 1 장에서 라이브러리를 설치하는 방법은?

- ① 필요한 이름을 직접 나열해 한 번에 설치
- ② 매 장마다 그 장에서 필요한 것만 설치
- ③ `pip install -r requirements.txt`

21. 저장소를 private 으로 고른 학생이 12 장에서 겪는 일은?

- ① 대시보드 배포 때 권한 승인 단계가 하나 더 있다
- ② 요금이 청구된다
- ③ 아무 차이 없다

22. 파이프라인 다섯 단계 중 '가공'의 설명으로 옳은 것은?

- ① 정리된 것을 한눈에 보이는 그림으로 만든다
- ② 주가 데이터를 어디션가 가져온다
- ③ 쌓인 것을 다루기 좋은 모양으로 정리한다

23. 이 수업에서 GitHub 계정이 '내 저장소' 말고도 필요한 이유는?

- ① 파이썬을 내려받을 때 쓴다
- ② 8 장 Turso DB 와 12 장 Streamlit 에 그대로 로그인한다
- ③ 텔레그램 봇을 만들 때 쓴다

24. 라이브러리 7 개 중 holidays 를 설치하는 이유는?

- ① 주말·공휴일을 걸러 그날 증시가 열리는지 판단하려고
- ② 나라별 시간대를 변환하려고
- ③ 공휴일마다 알림 문구를 바꾸려고

25. VS Code 확장에 대한 설명으로 옳은 것은?

- ① 확장을 하나도 깔지 않으면 파이썬 코드를 실행할 수 없다

- ② Code Runner 를 깔면 실행이 편해지므로 함께 설치하는 것이 좋다
- ③ Code Runner 는 설치하지 않는다 — 2 장에 만들 .env 를 찾지 못하는 문제가 생긴다

26. 이번 장에서 내 GitHub 저장소를 만드는 방법으로 옳은 것은?

- ① 저장소를 만든 뒤 받은 파일을 웹 화면에 끌어다 놓는다
- ② .gitignore 를 만든 뒤 VS Code 소스 제어에서 Publish to GitHub 를 누른다
- ③ GitHub 웹에서 New repository 를 눌러 미리 만들어 둔다

27. 소스 제어 패널에 버튼이 두 개 보인다. 눌러야 할 것과 그 이유로 옳은 것은?

- ① 아래 Publish to GitHub — 저장소 생성 · 첫 커밋 · 업로드가 한 번에 끝나니까
- ② 둘 중 아무거나 — 결과는 같다
- ③ 위 Initialize Repository — 저장소를 만드는 버튼이니까

28. 실수로 위쪽 Initialize Repository 를 눌렀다. 슬라이드가 안내하는 복구 방법은?

- ① VS Code 를 닫고 → 숨김 파일을 켜서 .git 폴더만 지우고 → 폴더를 다시 연다
- ② GitHub 웹에서 저장소를 만들어 연결한다
- ③ 폴더를 지우고 압축파일을 다시 푼다

29. 위쪽 Initialize Repository 를 먼저 눌렀을 때 생기는 문제로 슬라이드가 드는 것은?

- ① GitHub 계정이 잠긴다
- ② 브랜치 이름 맞추기와 첫 커밋이 건너뛰어져, 윈도우에서는 브랜치가 master 로 만들어지고 10 장 자동 실행이 돌지 않는다
- ③ 메인 폴더의 파일이 전부 지워진다

30. .gitignore 를 받은 압축파일에서 복사하지 않고 직접 만들게 하는 이유는?

- ① 파일을 복사하면 커밋이 되지 않기 때문에
- ② 압축파일 안의 .gitignore 는 내용이 다르기 때문에
- ③ 틀려도 아무 오류를 내지 않는 파일이라, 무엇을 막고 있는지 보고 넘어가야 하기 때문에

31. .env 가 아직 없는 1 장에서 .gitignore 를 먼저 만드는 이유는?

- ① .env 파일을 자동으로 만들어 주기 때문에
- ② 토큰이 생기기 전에 방어막을 세우려고 — 한 번 커밋되면 GitHub 이력에 남아 되돌릴 수 없다
- ③ .gitignore 가 있어야 저장소를 만들 수 있어서

32. 저장소를 만들기 전에 git config 로 이름과 이메일을 설정해 두는 이유는?

- ① GitHub 로그인을 대신해 주기 때문에
- ② 커밋에는 누가 했는지가 함께 기록되기 때문에 — 안 하면 Author identity unknown 으로 막힌다
- ③ 저장소 이름이 그 이름으로 만들어지기 때문에

33. 메인 프로젝트 폴더 이름을 영문.숫자로 지으라고 하는 이유는?

- ① 한글 폴더에서는 파이썬이 실행되지 않아서
- ② 한글 이름은 GitHub 에서 검색되지 않아서
- ③ 폴더 이름이 저장소 이름으로 그대로 제안되는데, 한글은 한 글자당 하이픈 하나로 바뀌어서

34. 커밋 메시지를 적지 않고 Commit 버튼을 먼저 누르면 어떻게 되는가?

- ① 메시지 없이 그대로 커밋된다
- ② 커밋이 취소되고 아무 일도 일어나지 않는다
- ③ COMMIT_EDITMSG 가 열리고 소스 제어 패널이 잠긴 것처럼 보인다 — Git 이 기다리는 중이다

35. 1 장에서 GitHub 저장소부터 만들어 두는 이유는?

- ① 코드를 잃어버렸을 때 복구하려고
- ② 과제 제출을 GitHub 으로 받기 때문에
- ③ 10 장에서 GitHub 가 내 코드를 대신 실행하려면 그 코드가 거기 있어야 하기 때문에

36. 받은 watchlist.csv 를 열어 본 내용이다. 첫 줄 code,name 의 역할은?

```
code,name
005930,삼성전자
000660,SK 하이닉스
005380,현대자동차
```

- ① 첫 번째 종목

② 각 열의 이름(제목 줄)

③ 파일 형식 표시

37. .gitignore 를 열면 맨 위에 아래 세 줄이 있다. 이것이 뜻하는 것은?

```
# 비밀정보 - 절대 커밋하지 말 것. 한 번 커밋되면 GitHub 이력에 남아 되돌릴 수 없고,  
# 지금 private 이어도 나중에 public 으로 바꾸는 순간 그대로 공개된다. (FAQ 2 절 18 번)  
.env
```

① .env 파일을 자동으로 만들어 준다

② .env 파일을 GitHub 에 올리지 않는다

③ .env 파일을 삭제한다

38. 학기말까지의 로드맵에 있는 기능이 아닌 것은?

① 봇이 내 폰에 말을 건다

② 종가가 DB 에 쌓인다

③ 가만히 있어도 원하는 정보가 온다 (자동 실행)

④ 핸드폰에서는 정보 확인이 불가능하다

39. 터미널에 아래 두 줄을 차례로 쳤다. 첫 줄과 둘째 줄이 각각 하는 일은?

```
git config --global user.name  
git config --global user.name "hong-gildong"
```

① 첫 줄은 이름을 지우고, 둘째 줄은 이름을 넣는다

② 첫 줄은 지금 설정된 이름을 확인하고, 둘째 줄은 이름을 설정한다

③ 둘 다 같은 일을 한다

40. 직접 만든 .gitignore 의 앞부분이다. 이 목록 때문에 2 장에서 .env 를 만들고 나서 소스 제어 패널에서 보게 되는 것은?

```
# 비밀정보 - 절대 커밋하지 말 것  
.env  
.env.*  
!.env.example  
*token*.txt  
*secret*
```

① Changes 목록에 .env 가 뜨지만 커밋은 되지 않는다

- ② Changes 목록에 .env 가 아예 뜨지 않는다
- ③ .env 를 저장할 때마다 경고 창이 뜬다

41. 같은 .gitignore 의 전체 내용이다. 이 파일이 GitHub 에 올라가지 못하게 막는 것이 아닌 것은?

```
# 비밀정보 - 절대 커밋하지 말 것
.env
.env.*
!.env.example
*token*.txt
*secret*
__pycache__/
*.pyc
*.docx
*.pptx
*.pdf
*.jpg
*.jpeg
*.png
```

- ① .env
- ② __pycache__/ 폴더
- ③ watchlist.csv

42. 1 장에서 갖추는 개발 환경의 선후 관계를 설명한 것 중 잘못된 것은?

- ① Git 프로그램이 깔려 있어야 VS Code 소스 제어의 Publish to GitHub 가 동작한다
- ② VS Code 를 설치해야 그 안에 확장 Python(게시자 Microsoft)을 설치할 수 있다
- ③ Git 을 설치한 직후 VS Code 가 켜져 있어도 소스 제어가 바로 Git 을 찾는다

Ⅲ. 주관식 (12 문항)

43. 터미널에 그대로 치는 아래 한 줄로 설치되는 라이브러리는 모두 몇 개인가?

```
pip install requests python-dotenv pandas matplotlib holidays streamlit libsql-
client
```

44. 13 장 뒤 텔레그램으로 오게 될 메시지다. 빈칸에 들어갈 것은?

 삼성전자 (_____)
208,500 원 ▲ +1.24%

45. VS Code 왼쪽 탐색기에서 새 파일을 만들고 이름을 아래처럼 쳤다. 무엇이 잘못됐는가?

gitignore

46. 맥에서 아래를 쳤더니 버전 숫자 대신 설치 안내 창이 떴다. 어떻게 해야 하는가?

git --version

보기는 ㉠ ㉡ ㉢ ..., 항목은 (1) (2) (3) ... 으로 적었다. 짝 맞추기 문항의 답은 (1)-㉡ 형식으로 쓴다.

47. 이 프로젝트가 지나가는 다섯 칸을 순서대로 나열하시오.

㉠ 시각화 ㉡ 수집 ㉢ 확인 ㉣ 가공 ㉤ 저장

48. 아래 단계와 이 프로젝트에서 쓰는 기술을 짝지으시오.

(1) 수집 (2) 저장 (3) 시각화 (4) 확인

㉠ matplotlib ㉡ 네이버 금융 API ㉢ Turso DB (클라우드 SQLite) ㉣ 텔레그램 봇 · Streamlit
대시보드

49. VS Code 에서 내 GitHub 저장소를 만드는 다섯 단계를 순서대로 나열하시오.

㉠ 아래 public 을 마우스로 클릭한다
㉡ 소스 제어 패널을 열어 아래 Publish to GitHub 를 누른다
㉢ VS Code 로 메인 폴더를 열고 '신뢰함'을 고른다
㉣ 로그인 창에서 승인한다 — 생성 · 첫 커밋 · 업로드가 한 번에 끝난다
㉤ 저장소 이름이 영문·숫자인지 확인한다

50. 실수로 Initialize Repository 를 눌렀을 때의 복구 세 단계를 순서대로 나열하시오. (순서를 틀리면 복구되지 않는다)

㉠ 숨김 파일을 켜고 .git 폴더만 지운다
㉡ 폴더를 다시 연다
㉢ VS Code 를 닫는다

51. 2 장부터 매 장마다 반복하는 다섯 단계를 순서대로 나열하시오.

㉠ 파란 Commit 버튼을 누른다
㉡ 그 장의 zip 을 풀어 전부 복사해 메인 폴더에 덮어쓴다

- ㉔ 버튼이 Sync Changes 로 바뀌면 한 번 더 누른다
- ㉕ 소스 제어 패널을 열어 바뀐 파일 목록을 확인한다
- ㉖ 맨 위 한 줄 입력칸에 커밋 메시지를 쓴다

52. 이번 장에서 Git 관련으로 하는 네 가지를 하는 순서대로 나열하시오.

- ㉗ 소스 제어에서 Publish to GitHub 를 누른다
- ㉘ Git 프로그램을 설치한다
- ㉙ git config 로 이름과 이메일을 설정한다
- ㉚ VS Code 를 켜다 켜다

53. 아래 버튼과 그 버튼이 하는 일을 짝지으시오.

- (1) Initialize Repository (2) Publish to GitHub (3) Commit (4) Sync Changes
- ㉛ '여기까지 됐다'고 내 컴퓨터에 도장을 찍는다
 - ㉜ 내 컴퓨터 안에만 저장소를 만든다 — GitHub 과는 관계가 없다
 - ㉝ 커밋한 것을 GitHub 으로 올려 원격과 로컬을 맞춘다
 - ㉞ 저장소 생성 · 첫 커밋 · 업로드를 한 번에 한다

54. 아래 증상과 그 원인을 짝지으시오.

- (1) Author identity unknown 이라고 뜬다
- (2) 저장소 이름이 하이픈 범벅이 됐다
- (3) Publish to GitHub 버튼이 아예 없다
- (4) 로그인 창이 두 번 뜬다
- ㉟ 메인 폴더 이름에 한글이 들어 있었다
 - ㊱ git config 로 이름·이메일을 설정하지 않았다
 - ㊲ 윈도우에서는 원래 그렇다 — 만들기와 올리기가 서로 다른 두 번이다
 - ㊳ 폴더를 열 때 신뢰를 주지 않아 제한 모드로 열렸다

IV. 상황 진단 (12 문항)

증상 한 줄을 주고 원인과 조치를 짧게 적게 하는 문항이다. 원인과 조치를 모두 적어야 정답이다.

55. [증상] 윈도우에서 터미널에 `python --version` 을 쳤더니 `python` 을 찾을 수 없다고 나온다. 원인과 조치를 적으시오.

정답: 원인 — Python 설치 화면 맨 아래 Add python.exe to PATH 체크를 하지 않아 터미널이 `python` 을 찾지 못한다. / 조치 — Python 을 다시 설치하면서 그 체크박스를 켜다. (맥에서 같은 증상이면 원인이 다르다 — 명령 이름이 `python3` 이다)

근거: 「Python 설치」 / FAQ 2 절 11 번

56. [증상] `pip install -r requirements.txt` 를 쳤더니 그런 파일이 없다며 실패한다. 원인과 조치를 적으시오.

정답: 원인 — 1 장 누적본에는 `requirements.txt` 가 없다(2 장 누적본부터 생긴다). / 조치 — 라이브러리 이름을 그대로 나열해 한 번에 설치한다 (`pip install requests python-dotenv pandas matplotlib holidays streamlit libsql-client`, 맥은 `pip3`).

근거: 「라이브러리 7 개를 한 번에」 / FAQ 1 장 — `requirements.txt` 는 장마다 자란다

57. [증상] 터미널에서 `git --version` 을 쳤더니 `git` 을 찾을 수 없다고 나온다. 그대로 두면 이번 장 실습에서 무엇이 막히는지, 어떻게 조치해야 하는지 적으시오.

정답: 원인 — Git 프로그램이 아직 깔려 있지 않다. VS Code 의 소스 제어는 이 프로그램을 불러서 쓰기 때문에, 그대로 두면 Publish to GitHub 버튼이 아예 나타나지 않아 저장소를 만들 수 없다. / 조치 — 윈도우는 `git-scm.com` 에서 Git for Windows 를 받아 기본값 그대로 설치하고, 맥은 `git --version` 을 쳤을 때 뜨는 안내 창에서 Xcode Command Line Tools 를 설치한다. 설치한 뒤에는 VS Code 를 켜야 소스 제어가 Git 을 찾는다.

근거: 「Git 프로그램」 / 「이번 장에서 할 일」 / FAQ 2 절 22 번 — 켜져 있는 동안 깐 것은 인식되지 않는다

58. [증상] 저장소 이름을 친 뒤 그냥 엔터를 쳤더니, 만들어진 저장소가 `private` 이다. 왜 이렇게 됐는지와 앞으로 무엇이 달라지는지 적으시오.

정답: 원인 — 이름을 친 뒤 나오는 두 항목 중 위 `private repository` 가 기본 선택이라, 그냥 엔터를 치면 `private` 이 된다. / 조치 — 수업은 그대로 진행할 수 있다. 다만 12 장에서 대시보드를 배포할 때 '이 저장소를 봐도 된다'는 권한 승인 단계가 하나 더 있고, 승인 전에는 배포 화면의 저장소 목록에 아예 나타나지 않는다. 바꾸려면 저장소 Settings 맨 아래에서 `public` 으로 전환하면 된다.

근거: 「④ 아래 `public` 을 클릭합니다」 / 「GitHub 저장소(Repository)」 / FAQ 2 절 16 번

59. [증상] Publish to GitHub 를 눌렀더니 Author identity unknown / Please tell me who you are 라고 뜨며 멈춘다. 원인과 조치를 적으시오.

정답: 원인 — Git 에게 이름과 이메일을 알려주지 않았다. 커밋에는 누가 했는지가 함께 기록되기 때문에 설정이 없으면 커밋 자체가 되지 않는다. / 조치 — 터미널에서 `git config --global user.name "내_깃허브_아이디"` 와 `git config --global user.email "내_이메일"` 을 실행한다. 이메일은 GitHub 계정 이메일과 같게 쓰고, `--global` 이라 이 컴퓨터에서 한 번만 하면 된다.

근거: 「Git 에게 내가 누구인지 알려줍니다」 / FAQ 1 장 5-1 번 — 원인이 화면에 드러나지 않아서 미리 해 두는 것이다

60. [증상] 소스 제어 패널을 열었는데 안내에 적힌 두 버튼이 없고 Manage Workspace Trust 만 보인다. 원인과 조치를 적으시오.

정답: 원인 — 폴더를 처음 열 때 뜨는 '이 폴더의 파일 작성자를 신뢰합니까'에서 신뢰를 주지 않아 제한 모드로 열렸다. 제한 모드에서는 git 기능이 통째로 꺼지는데 그 이유가 화면에 나오지 않는다. / 조치 — Manage Workspace Trust 를 누르거나 파일 → 작업 영역 신뢰 관리에서 신뢰함을 고른다. 신뢰를 줬는데도 버튼이 없으면 Git 이 안 깔린 것이니 `git --version` 으로 확인하고, 설치한 뒤 VS Code 를 꺾다 켜다.

근거: 「① VS Code 로 메인 폴더를 엽니다」 / 「Git 프로그램」 / FAQ 2 절 22 번

61. [증상] 저장소를 브라우저로 열어 봤더니 이름이 ----- 이다. 원인과 조치를 적으시오.

정답: 원인 — 메인 폴더 이름이 한글이었다. GitHub 저장소 이름에는 한글을 쓸 수 없어 한 글자당 하이픈 하나로 바뀐다. 입력칸에는 한글이 그대로 보이고 바뀐 이름은 오른쪽 작은 회색 글씨에만 나와서 알아채기 어렵다. / 조치 — 저장소 Settings → Repository name 에서 영문 이름으로 Rename 하고, VS Code 에서 폴더를 닫았다 다시 열어 Sync Changes 가 정상 동작하는지 확인한다. 애초에 메인 폴더 이름을 영문·숫자로 지으면 생기지 않는다.

근거: 「③ 저장소 이름은 영문·숫자로」 / 「폴더는 두 개를 씁니다」 / FAQ 2 절 24 번

62. [증상] 소스 제어 패널에서 위쪽 Initialize Repository 를 눌러 버렸다. 무엇이 잘못됐고 어떻게 되돌리는지 적으시오.

정답: 원인 — 그 버튼은 내 컴퓨터 안에만 저장소를 만들어서 GitHub 쪽 작업(저장소 생성 · 업로드)이 통째로 건너뛰어진다. 브랜치 이름 맞추기와 첫 커밋도 건너뛰어져, 윈도우에서는 브랜치가 master 로

만들어지고 10 장 자동 실행이 아무 오류 없이 돌지 않게 된다. / 조치 — ① VS Code 를 닫고 ② 숨김 파일을 켜서 .git 폴더만 지우고 ③ 폴더를 다시 연다. 그러면 처음 상태로 돌아가 Publish to GitHub 가 다시 보인다.

근거: 「② 소스 제어 패널을 엽니다」 / FAQ 2 절 26 번 — 숨김 파일 켜기는 Finder ⌘⇧. / 탐색기 보기 → 숨긴 항목

63. [증상] 2 장에서 .env 를 만들었더니 소스 제어의 Changes 목록에 .env 가 보인다. 원인과 조치를 적으시오.

정답: 원인 — .gitignore 가 없거나 이름·내용이 잘못됐다(맨 앞의 점이 빠졌거나 .env 줄이 없다). 제대로 돼 있으면 .env 는 목록에 아예 뜨지 않는다. / 조치 — 커밋하기 전에 메인 폴더의 .gitignore 를 열어 파일 이름과 .env 줄을 확인해 고친다. 한 번 커밋되면 GitHub 이력에 남아 되돌릴 수 없고, private 이어도 나중에 public 으로 바꾸는 순간 공개된다.

근거: 「왜 .gitignore 가 먼저인가요」 / 「.gitignore 를 직접 만듭니다」 / FAQ 2 절 18 번

64. [증상] 맥에서 Finder 로 메인 폴더를 열었는데 방금 만든 .gitignore 가 보이지 않는다. 원인과 조치를 적으시오.

정답: 원인 — 맥은 이름이 점(.)으로 시작하는 파일을 숨긴다. 파일이 없어진 것이 아니다. / 조치 — Finder 에서 ⌘⇧. 을 누르면 보인다. VS Code 탐색기에서는 그냥 보이므로 거기서 확인해도 된다. (윈도우는 점과 상관없이 보이지만, .git 폴더는 hidden 속성이 붙어 있어 탐색기 보기 → 숨긴 항목을 켜야 보인다)

근거: 「.gitignore 를 직접 만듭니다」 / 「② 소스 제어 패널을 엽니다」 / FAQ 1 장 7 번 — 맥에서 ⌘A 로 전체 선택해 복사하면 이 파일이 조용히 빠지는 것도 같은 이유다

65. [증상] Commit 을 눌렀더니 COMMIT_EDITMSG 라는 낯선 파일이 열리고 소스 제어 패널이 먹통이 된 것 같다. 원인과 조치를 적으시오.

정답: 원인 — 커밋 메시지를 적지 않고 Commit 을 눌러서, Git 이 '그러면 여기에 적어라'며 편집기를 띄우고 기다리는 중이다. 고장이 아니다. / 조치 — 열린 파일 1 번 줄에 메시지를 적고 저장(Ctrl+S / ⌘S) → 탭 닫기(Ctrl+W / ⌘W). 닫는 순간 커밋이 끝난다. 다음부터는 맨 위 한 줄 입력칸에 메시지를 먼저 쓰고 Commit 을 누른다.

근거: 「메시지를 먼저 쓰고 Commit 을 누릅니다」 / FAQ 2 절 23 번

66. [증상] Commit 을 눌러 Changes 목록이 비었는데, 브라우저로 GitHub 저장소를 열어 보니 바뀐 파일이 반영돼 있지 않다. 원인과 조치를 적으시오.

정답: 원인 — 커밋은 '내 컴퓨터에 도장을 찍은 것'까지다. 아직 Sync Changes 를 누르지 않아 원격(GitHub)에는 가지 않았다. / 조치 — 커밋 뒤 Sync Changes 1 ↑ 로 바뀐 버튼을 한 번 더 누르고, 확인 대화상자에 OK 한다. 그때 원격과 로컬이 맞춰진다.

근거: 「Sync Changes 를 눌러야 GitHub 으로 갑니다」 / 「가장 중요한 개념」 — 원격과 로컬은 별개의 사본이다